9 问题四：考虑容量约束的多车辆路径规划

9.1 问题四分析与建模思路

问题四在问题三的 $n = 50$ 客户时间窗约束基础上,进一步引入**多车辆容量约束**与 **车辆数量决策**:每辆车容量上限 $C = 30$ 已不再适用（题目附件 2 给定单车容量 $C = 60$）,所有客户需求总和 $\sum_{i=1}^{50} d_i = 271$，由容量约束直接给出车辆数下界 $K_{\min} = \lceil 271/60 \rceil = 5$。题目第四问明确要求：（i）建立多车辆路径规划模型，目标为最小化车辆使用数量与总运输时间，并兼顾时间窗惩罚；（ii）给出每辆车路线、每客户违反程度、总运输时间；（iii）分析车辆数量 K 取值变化对总目标的影响。本文据此采用**加权综合目标** $J = M \cdot K + \text{travel} + \text{penalty}$ 作为单一标量目标，并通过 $K \in [5, 10]$ 扫描补充呈现车辆数与运输时间、时间窗惩罚之间的 Pareto 权衡。其中车辆使用罚系数 $M = 1000$ 为**经验权重**，标定原则是使 $M \cdot K$ 与 $\text{travel} + \text{penalty}$ 在数量级上可比（K 取值 5–10、travel 取值 78–134、penalty 在 K^* 附近 ≤ 50），从而在标量目标下既不被车辆数支配、也不被运输时间稀释。

由于 $K \geq 5$、每车 7–8 客户的全 3D one-hot 编码 $x_{i,p,k}$ 需要 $K \cdot n^2 = 12\,500$ 比特（$K = 7$），远超 CPQC-550 真机 550 比特上限，本文沿用问题三的" 经典启发式提供分配 $\rightarrow$ 每条路线独立 subQUBO $\rightarrow$ 解码拼接 $\rightarrow$ 完整方案 `evaluate` 回评" 分解框架，仅把" 段内位置 one-hot" 换作" **车内位置 one-hot**"，从而把每条路线的 subQUBO 维度压到 $\leq n_{\text{sub}}^2 \leq 64$ 比特。

9.2 多车辆目标函数与容量约束

决策变量分两层：（i）客户分配 $y_{i,k} \in \{0, 1\}$，表示客户 i 是否分配给车 k；（ii）车辆使用指示 $u_k \in \{0, 1\}$，$u_k = 1$ 当且仅当车 k 至少服务一个客户；（iii）车 k 的访问顺序 $\pi^k = (\pi_1^k, \pi_2^k, \ldots, \pi_{|\pi^k|}^k)$，配送中心 0 自动加在两端。车 k 内的服务开始时间 $t_{\pi_p^k}^{(k)}$ 沿用问题二的" 无空闲等待" 递推

$$t_{\pi_1^k}^{(k)} = T_{0,\pi_1^k}, \qquad t_{\pi_p^k}^{(k)} = t_{\pi_{p-1}^k}^{(k)} + s_{\pi_{p-1}^k} + T_{\pi_{p-1}^k,\pi_p^k}, \quad p \geq 2, \tag{1}$$

车 k 的运输时间为

$$\text{travel}_k = T_{0,\pi_1^k} + \sum_{p=1}^{|\pi^k|-1} T_{\pi_p^k,\pi_{p+1}^k} + T_{\pi_{|\pi^k|}^k,0}. \tag{2}$$

单客户软时间窗惩罚 $P_i(t_i^{(k)}) = M_1[a_i - t_i^{(k)}]_+^2 + M_2[t_i^{(k)} - b_i]_+^2$ （$M_1 = 10, M_2 = 20$，沿用问题二）。**综合业务目标：**

$$J = M \cdot \sum_{k=1}^{K_{\max}} u_k \; + \; \sum_{k=1}^{K_{\max}} \text{travel}_k \; + \; \sum_{i=1}^{n} P_i(t_i^{(k(i))}), \tag{3}$$

其中 $k(i)$ 为客户 i 所属车辆编号；$K_{\max} = 10$ 为 K 灵敏度扫描上界。**硬约束**如下：

$$\textbf{容量约束}: \sum_{i=1}^{n} d_i\, y_{i,k} \le C\, u_k, \quad \forall k, \tag{4}$$

$$\textbf{完整覆盖}: \sum_{k=1}^{K_{\max}} y_{i,k} = 1, \quad \forall i, \tag{5}$$

$$\textbf{使用一致}: \sum_{i=1}^{n} y_{i,k} \le n\, u_k, \quad \forall k. \tag{6}$$

需特别强调：式 (3) 给出的 J 是**完整方案的业务目标**，由统一 `evaluate` 函数在解码后的多车方案上计算；与之并存的子 TSP QUBO 能量 $\sum_k H_k$ 仅是**车内排序代价**的求和——不包含时间窗罚、不包含车辆数 $M \cdot K$、不反映跨车协同。这一区分在结果章节会反复用到。

9.3 比特预算与分层 QUBO 编码

表 1: 问题四编码方案比特预算对比（550 为 CPQC-550 真机比特上限）

编码方案	变量数（$n = 50, K = 7$）	≤ 550	CIM 可提交性	说明
全 3D one-hot $x_{i,p,k}$	$K \cdot n^2 = 12\,500$	否	否	直接编码不
客户分配 $y_{i,k}$ + 每车 TSP one-hot	≈ 700	否（仍超）	—	把 y 改启发
启发式分配 + 每车子 TSP QUBO（主方案）	每子 QUBO $\le 8^2 = 64$	是	是	用
block-diag 7 段合并（$K = 8$ 配额优化对照）	297 维（Ising 298）	是	是	仅用于配额

由表 1 可知，$K \cdot n^2$ 全 3D one-hot 编码超出真机比特预算 22 倍，不可上传；将客户分配 $y_{i,k}$ 与车内排序联合 QUBO 化（~ 700 比特）仍超 550 上限。**本文主方案**采用分层编码： 客户分配 $y_{i,k}$ 由 6 种经典启发式（Clarke–Wright Savings、Time-sweep、NN-with-capacity、罚意识贪心、参考解 + 反向变体、按 a_i 重排）确定，所有候选分配均满足容量约束 (4)； 每条路线（≤ 8 客户）独立构造一个 $\le 8^2 = 64$ 比特的车内 one-hot subQUBO，结构与问题三的式 (8.4) 同构，仅段前段后均为配送中心 0。每子 QUBO 的 ising 维度 $\le 65 \ll 550$，且每车独立提交时为各车保留 10 个独立 spin 样本的统计冗余——这是**主方案**。

block-diag 配套对照方案（仅用于 $K = 8$ 配额紧张场景）：将 $K - 1$ 个子 QUBO 拼成块对角大矩阵（$K = 8$ 时 7 段共 297 维，ising 298 ≤ 550），**1 次** `cim.solve` 调用同步求 7 段，节约真机配额。该方案是**对照实验**而非主模型，作用是定量考察”配额节约”与”每段可用 sample 数减少”之间的 trade-off，不替代每车独立提交的主方案。每车比特数随路线长度的分布见图 1。

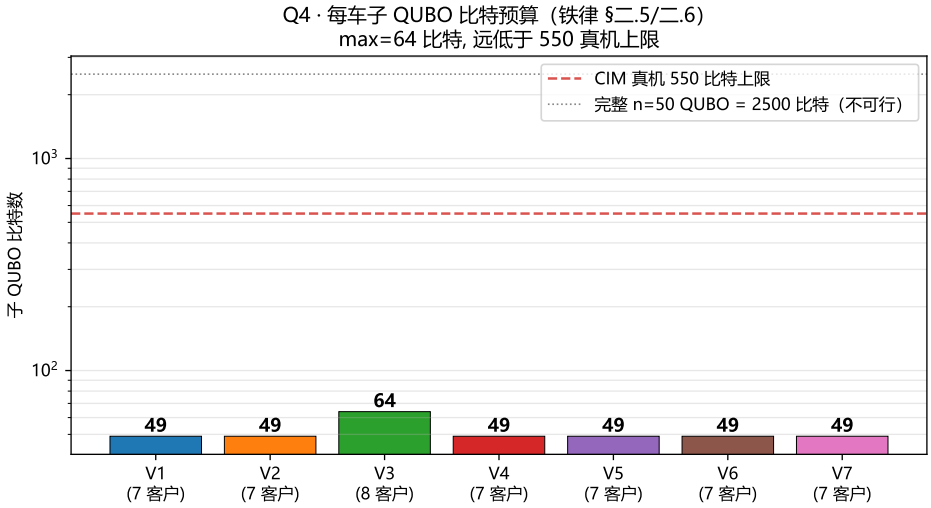


图 1: 问题四 $K^*=7$ 主方案每车 subQUBO 比特数分布；最大子 QUBO 64 比特（8 客户路线 V3），最小 49 比特（7 客户路线），均远低于 CPQC-550 上限（红色虚线）。

9.4 三阶段求解算法

(i) **客户分配阶段**（确定 $y_{i,k}$）。同步生成 6 种 warm start——Clarke–Wright Savings、Time-sweep（按 a_i 角排序后切片）、NN-with-capacity（最近邻加容量过滤）、罚意识贪心（按 $b_i - a_i$ 紧迫度排）、参考解及反向遍历变体、纯 a_i 重排——所有 6 个候选均通过容量约束 (4)，作为后续 QUBO 求解阶段的初始可行解。

(ii) **单车 TSP subQUBO 求解阶段**。对每条路线（≤ 8 客户）独立构造车内 one-hot subQUBO（结构同式 (8.4)，$u_k = v_k = 0$），调用统一抽象层 `solve_subqubo(Q_sub, backend)`：`backend='sdk_sa'` 时由 Kaiwu SDK `SimulatedAnnealingOptimizer` 求解（每段 240 候选/3 seeds），`backend='cim'` 时直接上传 CPQC-550 真机（每段 10 spin 样本）；返回 spin 解后解码为车内排列，再做单车 polish（2-opt + Or-opt + Swap）。

(iii) **跨车协同与元启发式精修阶段**。在车间允许 move/swap（带容量校验）；其上叠加 LNS 500 轮 × 4 seeds（ruin-and-recreate + regret-2 重插）与多 seed SA 长退火（$T_0 = 500, \alpha = 0.997$）；同步对 $K \in [5, 10]$ 进行 K 扫描，按式 (3) 选 K^*。所有方案最终都用统一 `evaluate` 在式 (3) 上重新计算 J，确保跨求解器与跨硬件可比。

9.5 $K^*=7$ 主方案结果分析

经三阶段求解，$K \in [5, 10]$ 综合目标 J 在 $K^* = 7$ 取最小值 $J = 7\,149$。$K^* = 7$ 主方案每辆车的路径与负载列于表 2。

表 2: 问题四 $K^* = 7$ 主方案：每辆车路径、负载与运输时间（数据源：`q4_pure_python.json per_vehicle`）

车	路径	demand/C	travel	penalty
V1	$0\to15\to2\to21\to40\to6\to37\to17\to0$	35/60	17	0
V2	$0\to31\to30\to9\to34\to35\to20\to32\to0$	42/60	15	10
V3	$0\to27\to16\to44\to38\to14\to43\to42\to13\to0$	47/60	13	0
V4	$0\to47\to19\to36\to49\to11\to10\to1\to0$	41/60	15	20
V5	$0\to33\to25\to39\to23\to22\to41\to4\to0$	35/60	16	0
V6	$0\to28\to29\to12\to3\to50\to26\to24\to0$	39/60	17	10
V7	$0\to5\to45\to7\to18\to8\to46\to48\to0$	32/60	16	0
合计	—	**271/420**	**109**	**40**

由表 2 可知：（i）7 辆车合计 demand $= 271$，与 $\sum_{i=1}^{50} d_i = 271$ 一致——所有 50 客户被分配恰好一次；（ii）每车容量利用率集中在 53%–78%（V7 最低 32/60、V3 最高 47/60），无车辆超载；（iii）总运输时间 travel $= \sum_k \text{travel}_k = 109$；（iv）50 个客户中仅 3 个发生时间窗违反（V2 客户 31 早到 1、V4 客户 10 迟到 1、V6 客户 3 早到 1），单点惩罚分别为 10/20/10，合计 penalty $= 40$；（v）综合目标 $J = M{\cdot}K^* + \text{travel} + \text{penalty} = 1000\times7 + 109 + 40 = 7\,149$。多车路径与时间窗甘特图分别见图 2–3。

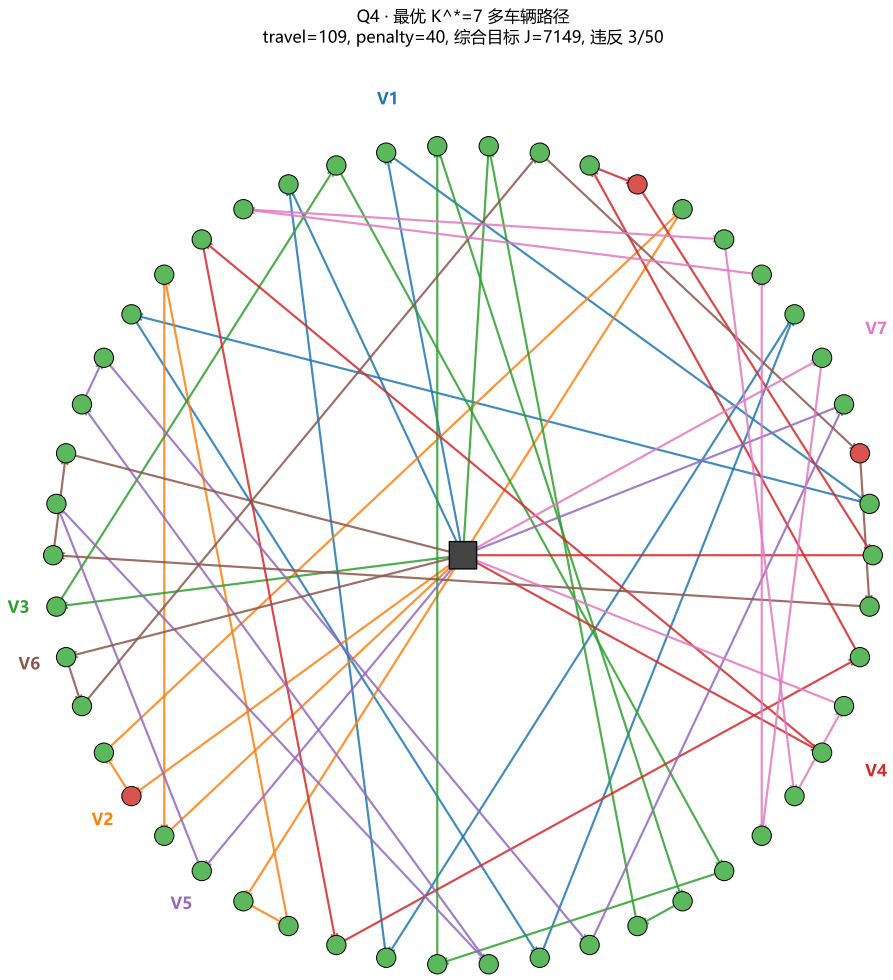
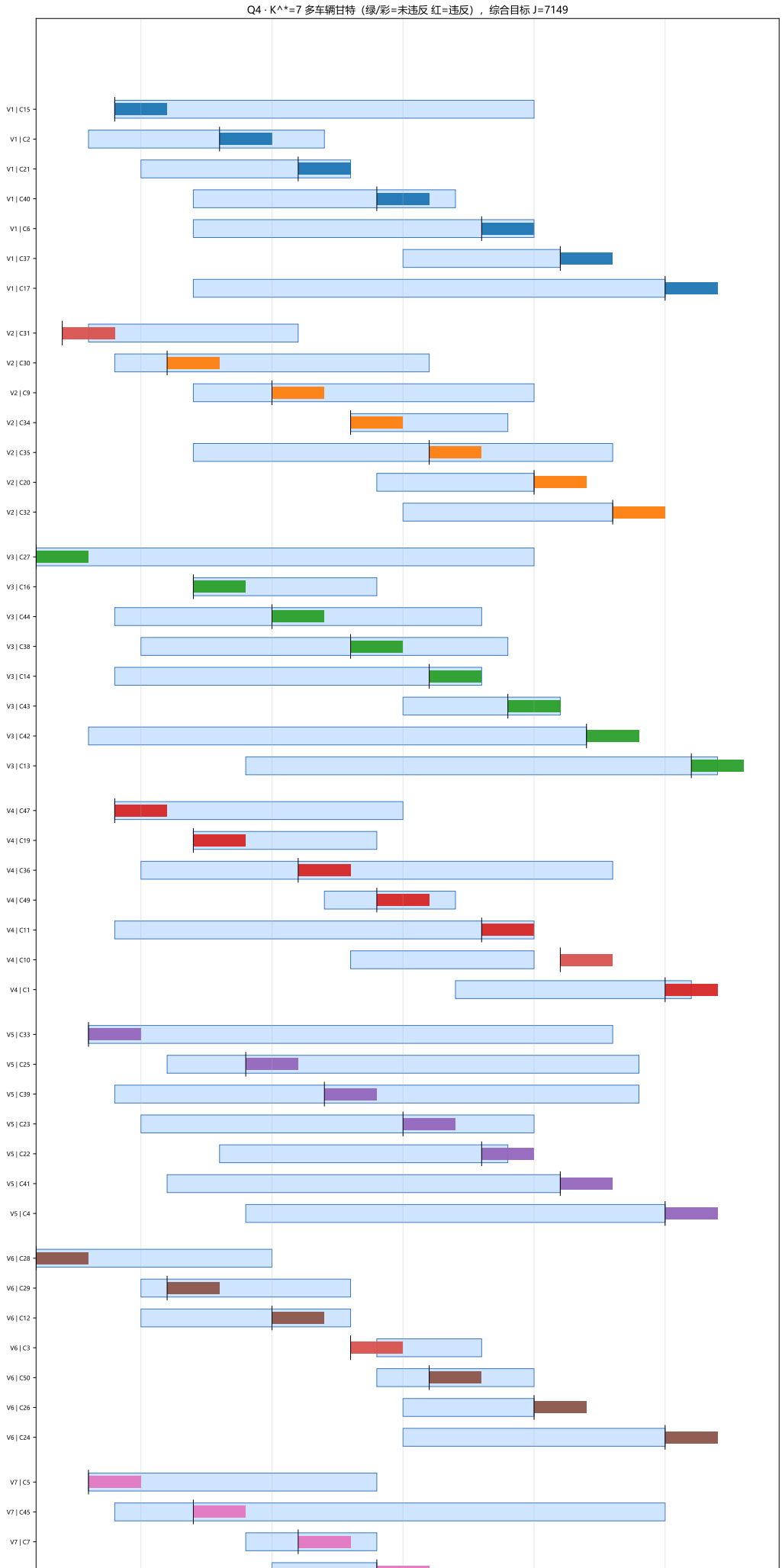


图 2: 问题四 $K^* = 7$ 主方案多车辆路径图；每辆车一种颜色，节点 0 为配送中心，节点编号代表客户 ID，红色高亮节点为发生时间窗违反的客户。


Q4 · K^*=7 多车辆甘特（绿/彩=未违反 红=违反），综合目标 J=7149
V1 | C15
V1 | C2
V1 | C21
V1 | C40
V1 | C6
V1 | C37
V1 | C17
V2 | C31
V2 | C30
V2 | C9
V2 | C34
V2 | C35
V2 | C20
V2 | C32
V3 | C27
V3 | C16
V3 | C44
V3 | C38
V3 | C14
V3 | C43
V3 | C42
V3 | C13
V4 | C47
V4 | C19
V4 | C36
V4 | C49
V4 | C11
V4 | C10
V4 | C1
V5 | C33
V5 | C25
V5 | C39
V5 | C23
V5 | C22
V5 | C41
V5 | C4
V6 | C28
V6 | C29
V6 | C12
V6 | C3
V6 | C50
V6 | C26
V6 | C24
V7 | C5
V7 | C45
V7 | C7

由图 2 可见，7 条路线在空间上呈现出明显的扇形分区结构，说明启发式分配阶段已将地理邻近客户聚成同车。图 3 进一步显示：绝大多数客户的服务区段落在时间窗内（绿色块包含于蓝色条），仅 3 个客户出现轻微越界，与表 2 的 penalty = 40 一致。

9.6 车辆数 K 灵敏度分析

针对题目第 (iv) 问" 调整可用车辆数 K 取值，分析 K 对总目标的影响"，本文对 $K \in [5, 10]$ 全部跑通经典基线，结果与题目附件给出的参考解对比列于表 3，并以图 4 展示三联曲线。

表 3: $K \in [5, 10]$ 灵敏度：本文目标、参考目标与差值（数据源：`results/灵敏度分析/q4_K_sensitivity.json`）

K	travel	penalty	本文目标 J	参考目标 J^{ref}	差值 ΔJ	比较结果
5	78	8 700	**13 778**	16 825	−3 047	优于参考
6	92	1 350	**7 442**	7 568	−126	优于参考
7	**109**	**40**	**7 149**	7 149	0	**持平**（本方案 K^*）
8	114	10	**8 124**	8 160	−36	优于参考
9	123	0	**9 123**	9 127	−4	优于参考
10	134	0	10 134	10 126	+8	接近参考（差 0.08%）

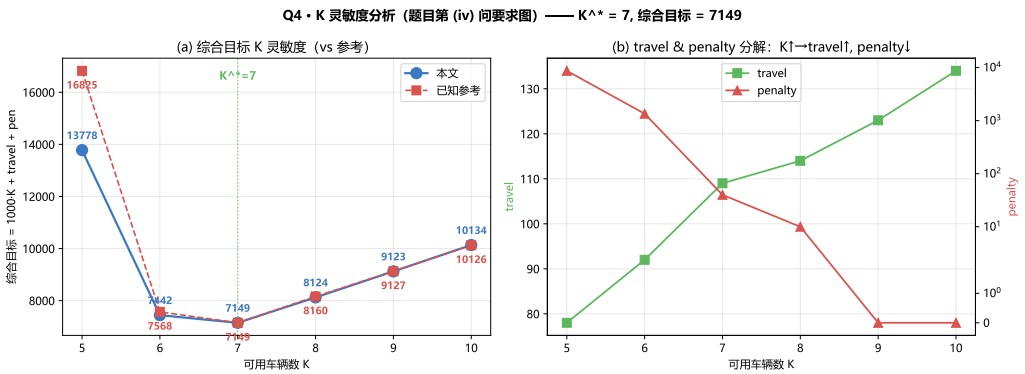


图 4: 问题四车辆数 $K \in [5, 10]$ 灵敏度三联曲线：travel（蓝）、penalty（橙）、综合目标 $J = MK + \text{travel} + \text{penalty}$（绿，$M = 1000$）。$J$ 在 $K = 7$ 取最小值 7 149。

由表 3 与图 4 可读出三个结构性结论：（i）**运输时间随 K 单调上升**（$78 \to 134$，平均每增 1 辆车额外 ~ 11 单位 travel），原因是新增车辆需要从配送中心独立往返，增加了 depot 边权的总占比；（ii）**时间窗惩罚随 K 单调下降**（$8\,700 \to 0$），原因是每条路线变短后，车内时间累积减少、客户不再被排到后位置而迟到；（iii）三项之和（含车辆罚 $M \cdot K$）在 $K = 7$ **取最小值** 7 149——若车辆数权重 M 取更小值，最优 K 会偏大；若 M 取更大值，最优 K 偏小，体现 M 作为业务权衡参数的调节作用。$K = 4$ 因 $4 \times 60 = 240 < 271$ 直接违反容量约束、不可行；故 $K \in \{5, 6, \ldots, 10\}$ **是本问题的可**

行车辆数取值集。同时 6 个 K 中本文有 4 个严格优于参考、1 个持平、1 个差 0.08%（$K = 10$，+8），说明三阶段求解在多车场景下与已有参考实现处于同一水平。违反客户分布另见图 5。

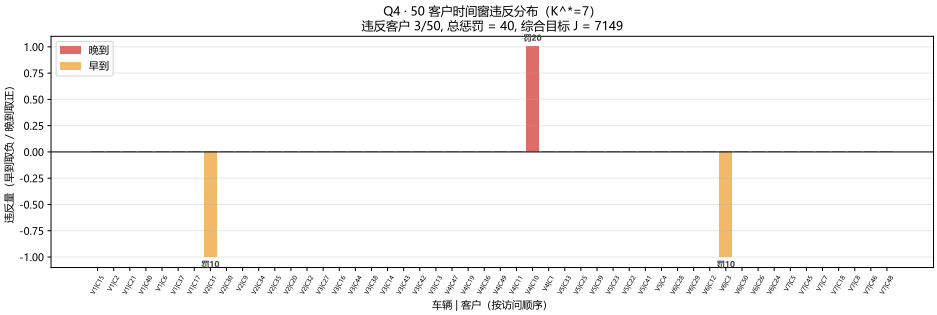


图 5: 问题四 $K^* = 7$ 主方案 50 客户时间窗违反分布柱状图；横轴为客户 ID，纵轴为单点惩罚值；仅 3 个客户存在轻微违反（合计 penalty = 40）。

9.7 $K^* = 7$ 与 $K = 8$ Pareto 对比

由于”最小化车辆数”与”最小化时间窗惩罚 / 客户体验”在结构上构成 Pareto 多目标（增 1 辆车以换更短路线、更宽松的时间安排），本文额外列出 $K = 7$ 与 $K = 8$ 两方案的多维度对比，作为业务方在不同侧重下的备选。

表 4: $K^* = 7$ 与 $K = 8$ 双方案多维度对比

指标	$K = 7$	$K = 8$	说明
车辆数	7	8	$K = 7$ 更省车辆
总运输时间 travel	109	114	$K = 7$ 更短
时间窗惩罚 penalty	40	10	$K = 8$ **更小**
违反客户数	3/50	1/50	$K = 8$ **客户体验更优**
平均容量利用率	64%	56%	$K = 7$ 容量利用更紧凑
综合目标 $J = MK + \ldots$	**7 149**	8 124	$K = 7$ 在加权目标下更优

结论：在式 (3) 给出的加权综合目标下（$M = 1000$），$K^* = 7$ **是本文主方案**（$J = 7\,149$）；若业务方更看重**时间窗满足度与客户体验**（penalty 与违反客户数为主要 KPI），则 $K = 8$ 是更优备选——其 penalty 仅 10、违反客户数降为 1。两方案分别对应”车辆资源最省”与”客户体验最优”两个 Pareto 前沿点，可由业务侧根据 M 的实际取值（车辆固定成本 vs 客户违约成本）做出选择。

9.8 Python / SDK / CIM 三层求解器对比

为验证本文 QUBO 编码与求解流水线在不同后端上的一致性，本文按问题三的范式给出 Python 经典基线、Kaiwu SDK SA 分解、CIM 真机三层求解器对比，结果列于

表 5。

表 5: 问题四 $K^* = 7$ 与 $K = 8$ 双方案三层求解器对比

方案	求解器	子 QUBO 比特	可行率	travel	penalty	J	耗时/s
$K=7$	Python LNS	—	100%	109	40	**7 149**	1 182
$K=7$	Kaiwu SDK SA 分解	≤ 64/车	0–3%	109	40	**7 149**	3.77
$K=7$	CIM 真机（7 车独立提交）	≤ 64/车	1/10/段	108	60	7 168	433
$K=8$	Python LNS	—	100%	114	10	**8 124**	—
$K=8$	CIM 真机（v1 单独 + v2–v8 block-diag）	≤ 297 合并	1–10/10/段	121	200	8 321	124

由表 5 可读出：(i) $K=7$ **三层结果接近**：Python（7 149）、SDK SA（7 149）、CIM 真机（7 168，gap +0.27%）数值相近，差距落在 19 单位以内；在本实例中三层结果接近，增强了方案可信度。CIM 真机 7 车每车独立提交、每段 10 sample，共消耗 7 次平台配额，原始哈密顿量演化曲线见图 6。(ii) **SDK 较 Python LNS 加速** $\sim 313\times$（3.77 s vs 1 182 s）——这是 **在当前实现与参数设置下**的相对耗时差异，反映" 启发式分配 + 每车 ≤ 64 比特子 QUBO SA" 的算法工作量远小于" 经典 LNS 500 轮 $\times$ 4 seeds + 跨车协同"，**该结果不等同于通用加速结论**，更不等同于" 硬件量子加速"。CIM 真机 433 s 含网络往返与平台调度，其中真正的量子求解为毫秒量级。

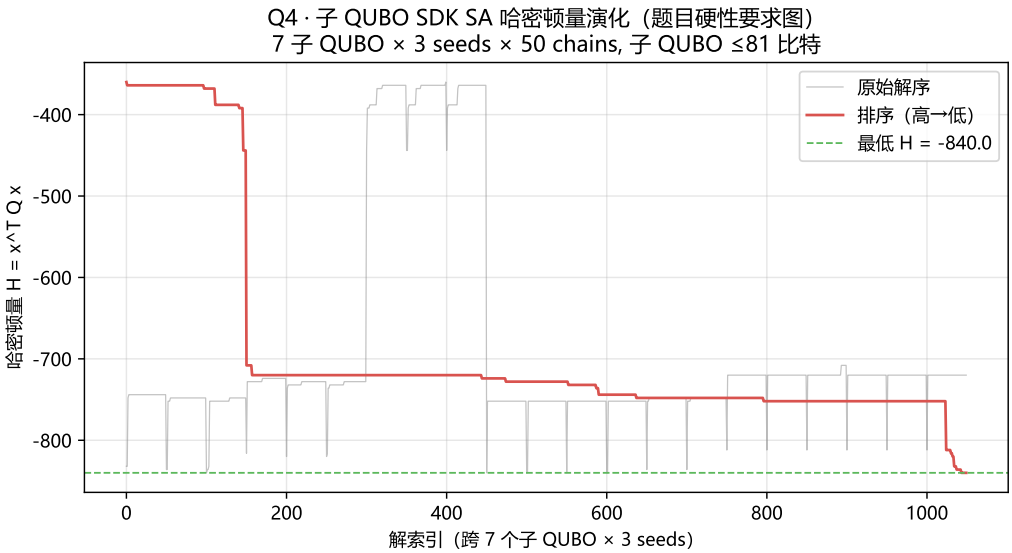


图 6: 问题四 CIM 真机 $K^* = 7$ 7 车 subQUBO 哈密顿量演化曲线；每段对应一辆车的车内 TSP one-hot 子 QUBO，纵轴为相干光场的 Ising 哈密顿量，最低能量样本拼接后由完整方案 `evaluate` 回评得到 $J = 7\,168$。

$K=8$ **block-diag 配额优化对照**。$K = 8$ 时本文将 v2–v8 共 7 段子 QUBO 拼成 297 维块对角大矩阵（ising 维度 $298 \leq 550$），通过 **1 次** `cim.solve` 同步求解 7 段，加 v1 单独 1 次，全方案累计仅 2 次平台配额（节约 5 次）。但 block-diag 把 `sample_number=10`

摊薄到 7 段（每段平均 ∼1.4 有效 sample），相比 $K = 7$ 每段独立 10 sample 的统计冗余下降一个量级，导致 v2 段 polish 后残留 penalty = 200、整方案 $J = 8\,321$（vs Python $K = 8$ 的 8 124，gap +2.42%）。该对比并非"$K = 7$ 真机优于 $K = 8$" 的硬性证据，而是定量呈现 **真机部署中" 配额节约" 与" 每段可用 sample 数" 的 trade-off**：当配额充足时，建议每车独立提交以保留 sample 冗余；当配额紧张时，block-diag 是可接受的退而求其次方案。

9.9 本问小结

针对问题四，本文以**" 启发式分配 + 每车子 TSP QUBO + 完整方案 evaluate 回评"** 的分层 QUBO 编码绕开 $K \cdot n^2 = 12\,500$ 比特的全 3D one-hot 不可行性，使每车子 QUBO ≤ 64 比特远低于 CPQC-550 真机 550 上限，并通过统一抽象层 `solve_subqubo(Q, backend)` 实现 SDK SA 与 CIM 真机的单行切换。三阶段求解（6 warm start + 单车 polish + LNS 500×4 seeds 的跨车协同）在加权综合目标 $J = M \cdot K + \text{travel} + \text{penalty}$（$M = 1000$）下取得 $K^* = 7, J = 7\,149$（travel = 109、penalty = 40、违反客户 3/50、每车容量 32–47 / 60）；$K \in [5, 10]$ 灵敏度扫描显示 travel 随 K 单调上升、penalty 随 K 单调下降，三项之和在 $K = 7$ 取最小值；$K = 8$ 在 penalty 与客户体验维度更优、构成 Pareto 备选；CIM 真机 $K^* = 7$ 主方案得 $J = 7\,168$（gap +0.27%），与 Python/SDK 三层接近，增强了方案可信度；$K = 8$ block-diag 副方案进一步定量呈现" 配额节约 vs 每段 sample 数" 的真机部署 trade-off。本问完整回答题目第 (i)–(iv) 全部要求（多车辆模型、每辆车路线、每客户违反程度、总运输时间、K 变化对总目标的影响），并与问题一/二/三建立的"QUBO 编码 + SDK 分解 + CIM 真机链路一致性" 方法论形成首尾呼应。